

API Documentation

Project: Enterprise Quantum Computing Platform — QFT Bank **Student:** Syed Muhammad Tahoor Ali **Diploma:** EduQual Level 6, Diploma in Artificial Intelligence Operations (ANPP-OP) **API version:** 0.1.0

1. Overview

The platform exposes a REST API built with FastAPI. All application endpoints are grouped under `/api/` by module. Interactive documentation (Swagger UI) is generated automatically by FastAPI and served live at `/docs`; a machine-readable OpenAPI schema is available at `/openapi.json`. This document is the human-readable reference.

Responses are JSON. Request bodies, where required, are JSON and are validated by Pydantic before any quantum or database work is performed, so malformed input returns a clear `422 Unprocessable Entity` with field-level detail rather than an internal error.

Base URL. In local development the API is served directly by Uvicorn at `http://localhost:8000`. In the Kubernetes deployment the browser never contacts the backend directly: the frontend (nginx) is exposed on NodePort `30080` and reverse-proxies every `/api/*` and `/health` path to the backend over the in-cluster network. The backend service itself is `ClusterIP`-only.

CORS. Cross-origin requests are permitted only from `http://localhost:5173` (the Vite development server). In the deployed configuration the frontend and API share an origin through the nginx proxy, so CORS does not apply.

2. Service Endpoints

Two endpoints sit outside the module routers.

`GET /` — service banner. Returns the service name, a running status, and a UTC timestamp.

`GET /health` — liveness check. Returns `{"status": "healthy", "storage_backend": "..."}` , where `storage_backend` is either `postgresql` or `in-memory`, reporting which persistence backend is currently active. This is the field to check when confirming whether the database connected successfully.

`GET /metrics` — Prometheus metrics exposition, added by the instrumentator. Scraped by Prometheus; not intended for direct human consumption.

3. Module 1 — Quantum Circuits

Prefix: `/api/quantum`

POST /api/quantum/execute

Builds a quantum circuit from an ordered gate list, executes it on the Qiskit Aer simulator, and returns measurement results plus a text circuit diagram.

Request body (`CircuitRequest`):

Field	Type	Constraints	Default	Meaning
<code>num_qubits</code>	int	1-20	—	Number of qubits (capped at 20 to protect low-RAM machines)
<code>gates</code>	array	at least 1	—	Ordered list of gate instructions
<code>shots</code>	int	1-10000	1024	Number of measurement repetitions
<code>measure_all</code>	bool	—	true	Measure all qubits at the end

Each entry in `gates` is a `GateInstruction`: `{"gate": "<name>", "qubits": [...], "params": [...]}`. `params` is only required for the rotation gates (`rx`, `ry`, `rz`) and carries the angle in radians.

The 13 supported gates are: `h`, `x`, `y`, `z`, `s`, `t` (single-qubit, no params); `rx`, `ry`, `rz` (single-qubit, one angle param); `cx`, `cz`, `swap` (two-qubit); and `ccx` (three-qubit Toffoli).

Example request (a Bell state):

```
{
  "num_qubits": 2,
  "gates": [
    {"gate": "h", "qubits": [0]},
    {"gate": "cx", "qubits": [0, 1]}
  ],
  "shots": 1024,
  "measure_all": true
}
```

Response (`CircuitResult`) contains: `execution_id`, `timestamp`, `num_qubits`, `gates_applied`, `shots`, `circuit_diagram_text` (an ASCII rendering of

the circuit), `counts` (measured bitstring frequencies), `probabilities` (normalised counts), and `execution_time_ms`.

GET /api/quantum/gates

Returns the catalogue of supported gates with their qubit and parameter requirements. Useful for building a UI or validating input client-side.

GET /api/quantum/history

Returns a list of `HistorySummary` objects (execution id, timestamp, qubit count, gate count, shots) for previously executed circuits.

GET /api/quantum/history/{execution_id}

Returns the full `CircuitResult` for a single past execution.

DELETE /api/quantum/history

Clears the circuit execution history.

4. Module 3 — Quantum Algorithms

Prefix: `/api/algorithms`

All four algorithms return the same wrapper (`AlgorithmResult: algorithm, execution_id, timestamp, result`), where `result` is an algorithm-specific object. A single rigid schema is deliberately avoided because the four algorithms produce genuinely different-shaped output.

POST /api/algorithms/grover

Grover's search over a space of $2^{\text{num_qubits}}$.

Field	Type	Constraints	Default
<code>num_qubits</code>	int	1-6	—
<code>marked_state</code>	string	—	—
<code>shots</code>	int	1-10000	1024

`marked_state` is the target bitstring, e.g. `"101"`.

POST /api/algorithms/qft

Quantum Fourier Transform applied to an encoded basis state.

Field	Type	Constraints	Default
<code>num_qubits</code>	int	1-6	—
<code>input_state</code>	string	—	—
<code>shots</code>	int	1-10000	1024

POST /api/algorithms/qaoa

QAOA solving Max-Cut on a supplied graph.

Field	Type	Constraints	Default
<code>edges</code>	array of <code>[int, int]</code>	—	—
<code>num_nodes</code>	int	2-6	—
<code>p_layers</code>	int	1-3	1
<code>shots</code>	int	1-5000	512
<code>max_iterations</code>	int	1-100	30

`p_layers` sets QAOA circuit depth (more layers = more expressive but slower); `max_iterations` bounds the classical COBYLA optimiser.

POST /api/algorithms/vqe

Variational Quantum Eigensolver.

Field	Type	Constraints	Default
<code>num_qubits</code>	int	1-6	—
<code>max_iterations</code>	int	1-200	50

GET /api/algorithms/history and /history/{execution_id}

List past algorithm runs, or fetch one full `AlgorithmResult` by id.

5. Module 2 — Portfolio Optimization

Prefix: `/api/optimization`

POST /api/optimization/portfolio

Runs the full portfolio optimization: simulated market data → QUBO → Ising Hamiltonian → QAOA (real COBYLA optimisation) benchmarked against an exact classical brute-force optimum.

Field	Type	Constraints	Default	Meaning
<code>num_assets</code>	int	3-8	—	Number of simulated assets (capped at 8 so brute force stays feasible)
<code>budget</code>	int	≥1	—	Exact number of assets to select (must be < <code>num_assets</code>)
<code>risk_aversion</code>	float	0.0-5.0	0.5	Higher weights variance more heavily against return
<code>penalty_weight</code>	float	0.1-10.0	2.0	Strength of the "select exactly <code>budget</code> " constraint
<code>shots</code>	int	64-5000	512	QAOA measurement shots
<code>max_iterations</code>	int	5-100	30	COBYLA iteration cap
<code>p_layers</code>	int	1-3	1	QAOA layers
<code>seed</code>	int	—	42	Seed for reproducible simulated market data

Response (`PortfolioOptimizationResult`): `execution_id` , `timestamp` , and a `result` object containing the problem setup, the classical solution, the quantum (QAOA) solution, and a comparison (whether the quantum solution matched the classical optimum, and the cost difference).

GET /api/optimization/history and /history/{execution_id}

List past optimizations, or fetch one full result by id.

6. Module 4 — Executive Dashboard

Prefix: `/api/dashboard`

These endpoints aggregate activity from the other modules; they run no new quantum computation.

`GET /api/dashboard/overview` — high-level summary of platform activity. `GET /api/dashboard/kpis` — executive KPIs (total experiments, success rates, portfolio statistics). `GET /api/dashboard/history` — merged execution history across all modules. `GET /api/dashboard/report` — the research progress report combining history and KPIs.

7. Module 5 — Post-Quantum Cryptography Readiness

Prefix: `/api/pqc`

`GET /api/pqc/algorithms/classical` — reference of current public-key algorithms and their quantum vulnerability. `GET /api/pqc/algorithms/nist-pqc` — the NIST PQC standards catalogue (ML-KEM, ML-DSA, SLH-DSA, HQC). `GET /api/pqc/threat-context` — contextual explanation of the quantum threat. `POST /api/pqc/threat-demo` — conceptual quantum-threat demonstration (reuses Module 3's QFT for phase estimation). `POST /api/pqc/inventory/scan` — simulated cryptographic inventory scan with risk scoring. `GET /api/pqc/risk-assessment` — organisational risk assessment. `GET /api/pqc/migration-plan` — the multi-phase migration plan. `GET /api/pqc/readiness-report` — consolidated executive readiness report.

8. Module 6 — Multi-Framework Demonstration

Prefix: `/api/frameworks`

`GET /api/frameworks/compare?shots=<n>` — constructs the same Bell state in Qiskit, PennyLane, and Cirq and returns the three results side by side, demonstrating framework-independent correctness. Satisfies the specification's PennyLane and Cirq requirements.

9. Module 7 — Classical ML vs Quantum

Prefix: `/api/ml`

POST /api/ml/compare

Trains a classical machine-learning model on simulated markets and compares its asset-selection against the quantum approach.

Field	Type	Constraints	Default
-------	------	-------------	---------

num_assets	int	3-8	—
budget	int	≥1	—
risk_aversion	float	0.0-5.0	0.5
penalty_weight	float	0.1-10.0	2.0
num_scenarios	int	20-200	60
eval_seed	int	—	42

Returns an `MLComparisonResult` (`execution_id`, `timestamp`, `result`).

GET /api/ml/history and /history/{execution_id}

List past comparisons, or fetch one by id.

10. Market Analytics

Prefix: `/api/analytics`

Three complementary views of a simulated market, each answering a different question.

`GET /api/analytics/market/{num_assets}?seed=<n>` — a Pandas-structured summary table of the market with per-asset return, volatility, mean correlation, and return/risk ratio, plus summary statistics.

`GET /api/analytics/market/{num_assets}/chart?seed=<n>` — a static risk/return scatter rendered server-side by Matplotlib, returned as a base64-encoded PNG for embedding in reports.

`GET /api/analytics/market/{num_assets}/interactive?seed=<n>` — the same data as an interactive Plotly figure (JSON), which the frontend hydrates into a chart supporting hover, zoom, and pan.

`num_assets` is constrained to 2-20; `seed` defaults to 42 and makes the market reproducible.

11. Error Handling

Validation failures return `422` with a body describing which field failed and why (produced by Pydantic/FastAPI). Out-of-range values — for example a portfolio `budget` not smaller than `num_assets`, or a circuit exceeding 20 qubits — are rejected before any computation. Requests for a non-existent `execution_id` return `404`. Successful requests return `200`.

12. Notes for the Examiner

- The full, always-current endpoint list is generated by FastAPI itself and can be browsed interactively at `/docs` while the backend is running. This document mirrors that surface in prose.
- Every request schema enforces bounds chosen for this deployment: qubit and asset counts are capped so simulations remain feasible on modest hardware and so classical baselines stay exactly computable.
- Persistence is transparent to the API: whether a result is written to PostgreSQL or the in-memory fallback, the response shape is identical. The active backend is reported by `/health`.